

Day 1 종합실습_주연수

비동기 데이터 수집 파이프라인 실행 결과 보고서

프로젝트명	Day 1 데이터 수집 미니 파이프라인
캠퍼스 / 반	판교/7반
작성자	주연수
작성일	2026년 8월 6일

1. 프로젝트 개요

Open-Meteo, countries.dev, ip-api의 공개 API 3개를 비동기로 동시에 수집하고 Pydantic v2로 검증한 후 CSV와 Parquet로 저장하여 성능을 비교했다.

2. 실행 환경

항목	내용
운영체제	macOS
Python	3.11.15
주요 패키지	httpx, pydantic, pandas, pyarrow, pytest, ruff
GitHub	https://github.com/wndustn/skala-python

3. 프로그램 실행 결과

- API 3개 수집 결과 -> 완료
- Pydantic 검증 결과 -> 완료
- CSV와 Parquet 저장 결과 -> 완료
- 저장 결과 재로딩 검증 -> 완료
- 읽기·쓰기 성능 측정 결과 -> 완료

```

Last login: Thu Aug  6 19:14:22 on ttys000
→ 종합실습 git:(main) ✕ source .source .venv/bin/activate
[python -m app.main
=== 1. 공개 API 3개 동시 수집 ===
수집 완료: weather=1, country=1, ip=1

=== 2. Pydantic v2 스키마 검증 ===
검증 완료: weather, country, ip

=== 3. 데이터 결합 ===
결합 완료: 72시간

=== 4. CSV / Parquet 저장 및 성능 비교 ===
CSV: 0.003958초, 9535 bytes
Parquet: 0.993955초, 8504 bytes

=== 5. 저장 결과 재검증 ===
재로딩 완료: CSV=72건, Parquet=72건
CSV 읽기 시간: 0.001439초
Parquet 읽기 시간: 1.507958초

전체 파이프라인이 정상 완료되었습니다.
(.venv) → 종합실습 git:(main) ✕

```

그림 1. python -m app.main 실행 결과

확인 결과: API 3개가 정상 수집·검증되었고, 서울 3일 날씨 72시간 데이터가 CSV와 Parquet 각각 72행으로 저장된 뒤 동일하게 재로딩 되었습니다.

4. 테스트와 코드 품질

4.1 pytest

```

Last login: Thu Aug  6 19:20:23 on ttys000
[→ 종합실습 git:(main) ✕ source .venv/bin/activate ]
[ (.venv) → 종합실습 git:(main) ✕ pytest -v ]

===== test session starts =====
platform darwin -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /Users/jooyeonsu/Desktop/Skala_자료모음/08.06/종합실습/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/jooyeonsu/Desktop/Skala_자료모음/08.06/종합실습
configfile: pyproject.toml
testpaths: tests
plugins: asyncio-1.4.0, anyio-4.14.2
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 5 items

tests/test_api_client.py::test_fetch_all_data_calls_three_apis PASSED [ 20%]
tests/test_models.py::test_weather_rejects_out_of_range_probability PASSED [ 40%]
tests/test_models.py::test_ip_rejects_string_latitude PASSED [ 60%]
tests/test_pipeline.py::test_build_records PASSED [ 80%]
tests/test_storage.py::test_save_and_verify PASSED [100%]

===== 5 passed in 0.48s =====

```

그림 2. pytest -v 실행 결과 — 5개 테스트 통과

4.2 Ruff

```

[ (.venv) → 종합실습 git:(main) ✕ ruff check . ]
All checks passed!

```

그림 3. ruff check . 실행 결과 — 오류 없음

4.3 Git 커밋 이력

```

[ (.venv) → 종합실습 git:(main) ✕ git log --oneline ]

e85c0cd (HEAD -> main, origin/main) fix: Python 3.11 실행 호환성 개선
12150fd docs: 제출용 Git 이력 추가
1e6a6f3 feat: CSV와 Parquet 읽기 성능 측정 추가
2a07dbb feat: Day 1 데이터 수집 파이프라인 완성
~
~

```

5. CSV와 Parquet 성능 비교

형식	쓰기 시간	읽기 시간	파일 크기	의견
CSV	0.004800초	0.002144초	9,535 bytes	사람이 직접 읽기 쉽다.
Parquet	0.794823초	2.320098초	8,504 bytes	자료형 보존과 압축에 탁월하다.

측정 해석:

72행의 데이터를 저장했다. CSV의 읽기와 쓰기 속도가 Parquet보다 빨랐다. 반면 파일 크기는 Parquet가 CSV보다 조금 작았다. 측정 시간은 컴퓨터 상태나 실행 순서에 따라 달라질 수 있음을 유의해야 될 것 같다.

6. 본인 의견 및 개선 사항

- 구현하며 어려웠던 점:

처음에는 세 API에서 서로 다른 형태의 JSON이 반환되어 어떤 값을 골라 검증하고 저장해야 하는지 이해하는 것이 생각보다 까다로웠습니다. 특히 가상환경과 Python 버전이 맞지 않아 지속적으로 실행 오류가 발생했는데, 오류 메시지를 확인하고 환경을 다시 설정하면서 해결할 수 있었습니다.

- 비동기 수집의 장점:

API를 하나씩 차례로 호출하지 않고 세 요청을 동시에 시작할 수 있어 전체 대기 시간을 탁월하게 줄일 수 있었습니다. 이번 실습을 통해 `asyncio.gather()`가 여러 작업을 함께 실행하고 결과를 한 번에 받을 때 사용된다는 점을 이해할 수 있었습니다.

- CSV와 Parquet 비교:

이번 실행에서는 CSV의 읽기와 쓰기가 더 빨랐지만, 파일 크기는 Parquet가 조금 더 작았습니다. CSV는 파일을 직접 열어 내용을 확인하거나 수정하기 편하다는 장점이 있었습니다. 반면 Parquet는 데이터를 바로 확인하기는 어렵지만, 자료형을 유지할 수 있고 데이터의 용량을 줄이는 데 유리합니다. 따라서 데이터의 양과 사용 목적에 따라 적절한 저장 형식을 선택하는 것이 중요하다고 생각합니다.

- 개선할 사항:

현재 코드는 API 요청에 실패하면 오류 메시지를 출력하고 프로그램이 종료됩니다. 일시적인 네트워크 문제일 수도 있으므로, 일정 시간 후 자동으로 다시 요청하는 기능을 추가하면 프로그램을 더 안정적으로 실행할 수 있을 것 같습니다. 또한 이번에는 읽기와 쓰기 시간을 한 번씩만 측정했는데, 여러 번 실행한 후 평균값을 비교한다면 조금 더 신뢰할 수 있는 결과를 얻을

수 있을 것 같습니다.

- **코드 품질 개선 의견:**

기능별로 파일이 나누어져 있어 필요한 코드를 찾고 전체 처리 과정을 이해하는 데 도움이 되었습니다. `pytest`를 통해 주요 기능이 정상적으로 동작하는지 확인하고, `Ruff`를 통해 코드 형식과 작성 규칙도 점검할 수 있었습니다. 현재 작성된 테스트뿐만 아니라 잘못된 API 응답, 네트워크 오류, 빈 데이터와 같은 상황에 대한 테스트도 추가한다면 예상하지 못한 오류를 줄일 수 있을 것 같습니다. 앞으로는 비슷한 코드가 반복되는 부분도 조금 더 정리하여 읽기 쉽고 수정하기 편한 코드로 개선하고 싶습니다.